TP1.2 - Sudoku genérico

Gonçalo Faria e Ângelo Fernandes - Lógica Computacional, 2026/2027

Neste exercício implementámos um Sudoku genérico numa grelha $n^2 \times n^2$. Os números vão de 1 a n^2 e não se podem repetir nas linhas, colunas e blocos. As pistas ficam com o valor inicial. Usamos grupos de células para representar estas regras e o CP-SAT para procurar uma solução.

Procuramos uma solução válida. O enunciado não pede solução única.

Como montamos o modelo

Usamos `box(n)` para guardar um dicionário `(linha, coluna) -> valor ou None`. `cube` preenche um bloco e `path` um segmento reto, nos dois sentidos. Na matriz, zero significa que a célula está fora do grupo ou ainda não tem valor fixo.

`random_clues` escolhe posições sem repetição e valores independentes. Esses valores podem tornar o caso impossível. Nesse caso `solve()` devolve `None` e os exemplos tentam outras pistas, até 20 vezes.

R5 pede `AllDifferent` para todos os grupos, incluindo as pistas. Por isso, as pistas deste grupo também têm de ter valores diferentes entre si, mesmo que estejam em zonas diferentes da grelha. Com mais de n^2 pistas no mesmo grupo, não há solução. O sorteio não garante solução única.

Usamos CP-SAT porque permite escrever variáveis inteiras e `AllDifferent` diretamente, como nos exemplos da cadeira. `box`, `cube` e `path` só escolhem as células. A regra fica em `SudokuCSP.add_groups`.

Exemplo: n=2, grelha 4×4

Pistas geradas em 1 tentativa(s):

0 0 | 0 1
0 2 | 0 0
---
0 0 | 0 0
0 0 | 0 0
Solução validada:

4 3 | 2 1
1 2 | 4 3
---
2 1 | 3 4
3 4 | 1 2
Grupos: 13; estado: `OPTIMAL`; tempo do solver: 0.0022 s; ramos: 192.

Exemplo: n=3, grelha 9×9

Pistas geradas em 1 tentativa(s):

0 0 0 | 0 0 0 | 0 0 0
0 7 0 | 0 1 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
---
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
---
0 0 0 | 0 0 6 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
Solução validada:

5 1 8 | 6 9 7 | 2 4 3
4 7 3 | 2 1 5 | 9 6 8
6 2 9 | 3 4 8 | 1 5 7
---
9 6 5 | 7 8 4 | 3 2 1
1 8 7 | 5 3 2 | 4 9 6
3 4 2 | 1 6 9 | 7 8 5
---
2 3 1 | 4 5 6 | 8 7 9
7 9 6 | 8 2 1 | 5 3 4
8 5 4 | 9 7 3 | 6 1 2
Grupos: 28; estado: `OPTIMAL`; tempo do solver: 0.0229 s; ramos: 6493.

Teste adicional de escala: n=4, grelha 16×16 validada; 49 grupos; 0.1774 s e 46233 ramos nesta execução. Este tempo não garante o mesmo desempenho noutros puzzles.

Validação e caso impossível

`test_sudoku.py` tem cinco cenários: 4x4, 9x9, 16x16, diagonal e pistas impossíveis. Em cada solução conferimos linhas, colunas, blocos e pistas.

Incluímos também as verificações pedidas pelo enunciado: rejeitar uma coordenada e um valor inválidos, e percorrer caminhos ao contrário. Um caso impossível devolve `None`. Se o tempo acabar sem conclusão, o programa levanta um erro. Não podemos chamar impossível a um caso que o solver ainda não resolveu.

Validação: limites e caminhos corretos; as pistas contraditórias deram `INFEASIBLE`.

Extra: Sudoku diagonal

Juntamos duas `box`, uma por diagonal. O solver recebe-as como recebe qualquer outro grupo. Não foi preciso acrescentar outra regra ao modelo.

```
1 2 | 3 4
3 4 | 1 2
---------
4 3 | 2 1
2 1 | 4 3
```

Custo e limites

Com $N = n^2$, há N^2 variáveis e $3N$ grupos principais, cada um com N células. Criar as variáveis e os grupos custa $O(N^2)$ em tempo e espaço, sem contar o trabalho interno do solver. Uma matriz custa $O(N^2)$ e um bloco ou linha completos custam $O(N)$.

Resolver é a parte mais cara. Testar todas as atribuições daria até N^{N^2} possibilidades. CP-SAT usa restrições para cortar a procura, mas não há garantia de tempo polinomial. Os tempos acima são apenas medições destes exemplos. O teste 16x16 não garante resolver depressa qualquer grelha maior.

Registo de apoio

Incluímos o registo de apoio em `../conversas/TP1.2_assistencia.json`.